

Evidencia de ejecución — Tarea 1.2

Araceli Castillo · MAI 540: Machine Learning · Prof. Kevin A. Garcia Gallardo · 7 de septiembre de 2026

Este anexo reúne la evidencia del flujo obligatorio del README: entender el proyecto sin modificarlo, ejecutarlo en su estado inicial, solicitar y revisar un plan, autorizar la implementación y ejecutar de nuevo. Las capturas están en el orden en que ocurrieron.

modelo	accuracy	precision (1)	recall (1)	F1 (1)	fallas detectadas
ANTES	0.9112	0.0000	0.0000	0.0000	0 de 111
DESPUÉS	0.6856	0.1527	0.5586	0.2398	62 de 111

Figura 1

Paso 1 del flujo: Claude Code lee y explica el proyecto sin modificar ningún archivo.

mantenimiento_predictivo — * Análisis del proyecto completo — caffeinate · claude — 123x35

Qué modelo emplea actualmente (el "ANTES")

main.py es un punto de partida funcional pero limitado a propósito:

- Usa solo 4 variables numéricas: operating_hours, temperature_c, days_since_maintenance, energy_kw.

- train_test_split con test_size=0.25, random_state=42, stratify=y.

- Pipeline: SimpleImputer(strategy="median") → LogisticRegression(max_iter=1000, random_state=42).

- Sin escalado, sin manejo de categoricas, sin class_weight="balanced".

- Solo reporta accuracy y matriz de confusión.

Qué le falta para estar completo

Según el README, la implementación "DESPUÉS" debe:

1. Ampliar a los 10 predictores (añadir vibration_mm_s, pressure_bar, error_count_30d numéricas + las 3 categóricas).

2. Construir un ColumnTransformer con dos ramas:

- Numérica: SimpleImputer(strategy="median") + StandardScaler().

- Categórica: SimpleImputer(strategy="most_frequent") + OneHotEncoder(handle_unknown="ignore").

3. Mantener el train_test_split (test_size=0.25, random_state=42, stratify=y) y ajustar el preprocesamiento solo con train (evitar fuga de datos).

4. Cambiar el modelo a LogisticRegression(max_iter=1000, random_state=42, class_weight="balanced").

5. Reportar accuracy, precision, recall y F1 de la clase 1 (no solo accuracy).

6. Mostrar una comparación ANTES vs DESPUÉS con el mismo split.

7. Explicar con palabras propias qué cambió y por qué (por qué accuracy no basta con clase minoritaria; comentar recall/F1 de la clase 1).

Además, quedan pendientes los entregables: evidencia de ejecución antes/después y completar BITACORA.md. Fuera de alcance explícito: selección de variables, cross-validation, tuning, feature engineering avanzado y comparar varios algoritmos.

* Crunched for 26s · done 2:05 PM

✓ Update installed · Restart to apply

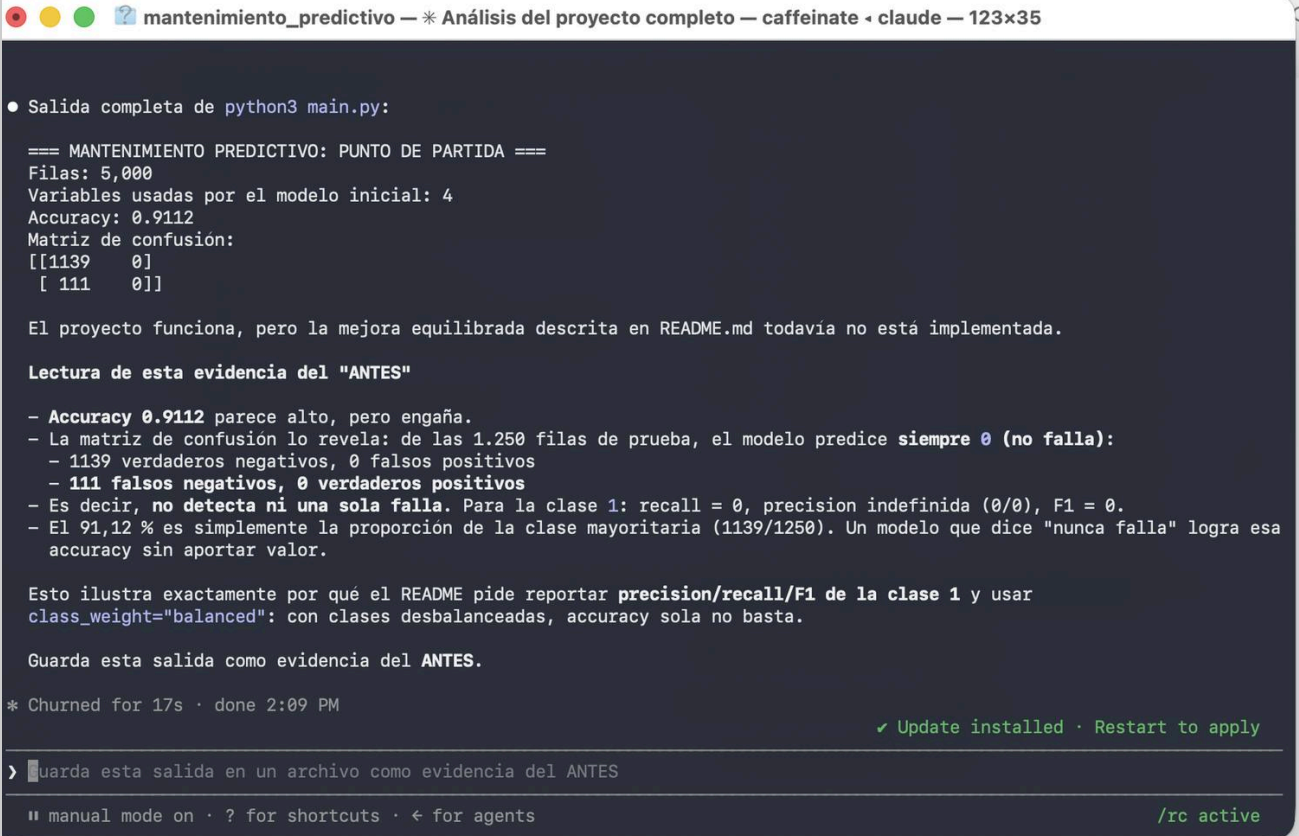
> ejecuta python main.py y guarda la salida del ANTES

▶ auto mode on (shift+tab to cycle) · ← for agents

/rc active

Figura 2

Paso 2: ejecución del proyecto en su estado inicial. Evidencia del ANTES.



```
mantenimiento_predictivo — * Análisis del proyecto completo — caffeinate < claude — 123x35

• Salida completa de python3 main.py:

=== MANTENIMIENTO PREDICTIVO: PUNTO DE PARTIDA ===
Filas: 5,000
Variables usadas por el modelo inicial: 4
Accuracy: 0.9112
Matriz de confusión:
[[1139   0]
 [ 111   0]]

El proyecto funciona, pero la mejora equilibrada descrita en README.md todavía no está implementada.

Lectura de esta evidencia del "ANTES"

- Accuracy 0.9112 parece alto, pero engaña.
- La matriz de confusión lo revela: de las 1.250 filas de prueba, el modelo predice siempre 0 (no falla):
  - 1139 verdaderos negativos, 0 falsos positivos
  - 111 falsos negativos, 0 verdaderos positivos
- Es decir, no detecta ni una sola falla. Para la clase 1: recall = 0, precision indefinida (0/0), F1 = 0.
- El 91,12 % es simplemente la proporción de la clase mayoritaria (1139/1250). Un modelo que dice "nunca falla" logra esa accuracy sin aportar valor.

Esto ilustra exactamente por qué el README pide reportar precision/recall/F1 de la clase 1 y usar class_weight="balanced": con clases desbalanceadas, accuracy sola no basta.

Guarda esta salida como evidencia del ANTES.

* Churned for 17s · done 2:09 PM

✓ Update installed · Restart to apply

> Guarda esta salida en un archivo como evidencia del ANTES

" manual mode on · ? for shortcuts · ← for agents /rc active
```

Figura 3

Paso 3: plan propuesto por Claude Code (pasos 4 a 8) antes de autorizar cambios.

```
Mismo que el ANTES, sin cambios:
train_test_split(X, y, test_size=0.25, random_state=42, stratify=y)
Así el conjunto de prueba es idéntico fila a fila y la comparación es justa.

Paso 4 – ColumnTransformer (dos ramas)

- Rama numérica (Pipeline): SimpleImputer(strategy="median") + StandardScaler()
- Rama categórica (Pipeline): SimpleImputer(strategy="most_frequent") + OneHotEncoder(handle_unknown="ignore")
- Combinar con ColumnTransformer([...]) aplicando cada rama a su lista de columnas.

Paso 5 – Pipeline final

Pipeline([
    ("prep", preprocesador),
    ("model", LogisticRegression(max_iter=1000, random_state=42, class_weight="balanced")),
])
model.fit(X_train, y_train) – el preprocesamiento se ajusta solo con train (lo garantiza el propio Pipeline). Sin tuning, sin CV, sin probar otros algoritmos.

Paso 6 – Evaluación

Sobre X_test:
- accuracy_score
- precision_score, recall_score, f1_score con pos_label=1 (o classification_report y leer la fila de la clase 1)
- confusion_matrix

Paso 7 – Comparación ANTES vs DESPUÉS

Para no perder el ANTES, la opción más limpia dentro del alcance:
- Envolver el modelo base (4 numéricas, sin balanceo, SimpleImputer mediana) y el modelo nuevo en dos bloques dentro del mismo script, con el mismo split.
- Imprimir una tabla comparativa: accuracy | precision(1) | recall(1) | F1(1) para ambos, mas ambas matrices de confusión.
- Alternativa aceptable: dejar la salida del ANTES ya guardada como evidencia y que main.py imprima solo el DESPUÉS citando los números del ANTES en comentarios. Recomiendo la tabla en el script porque el README pide "comparacion clara".

Paso 8 – Texto explicativo (en la salida y/o BITADORA.md)

Redactar con palabras propias:
- Tipos de variable: por qué numéricas se imputan con mediana + se escalan; por qué categóricas se imputan con moda + one-hot; qué hace handle_unknown="ignore".
- Fuga de datos: por qué fit solo con train.
- Desbalanceo: por qué accuracy sola no basta (el ANTES tiene 0.91 y recall 0 en clase 1); qué corrige class_weight="balanced"; interpretar recall (fallas detectadas) y F1 (equilibrio) de la clase 1.

Entregables afectados

- main.py reescrito.
- Evidencia de ejecución DESPUÉS (guardar salida).
- BITADORA.md completado con la explicación propia.
- data/DESCRIPCION.md dice 1.800 registros pero el CSV tiene 5.000 – fuera de alcance, solo lo menciono; no se toca salvo que lo pidas.

Fuera de alcance (no se hará)

Selección automática de variables, validación cruzada, búsqueda de hiperparámetros, feature engineering avanzado, comparación de múltiples algoritmos.

---
¿Autorizas la implementación con el Paso 7 en su versión "tabla comparativa dentro del script"?

* Baked for 23s · done 2:13 PM

✓ Update installed · Restart to apply

> [i], implementa la mejora con la tabla comparativa

" manual mode on · ? for shortcuts · ← for agents

/rc active
```

Figura 4

El plan completo guardado en plan.md para poder revisarlo entero.

```
mantenimiento_predictivo — * Análisis del proyecto completo — caffeinate - claude — 126x36

comparación de múltiples algoritmos.

---
¿Autorizas la implementación con el Paso 7 en su versión "tabla comparativa dentro del script"?

* Baked for 23s · done 2:13 PM

* recap: Goal: upgrade main.py for the predictive-maintenance classification task per README. I read the project, ran the
baseline (accuracy 0.9112 but recall 0 on failures), and gave you a full improvement plan. Next: you approve it so I can
implement. (disable recaps in /config)

> guarda el plan completo en plan.md

● Write(plan.md)
  └ Wrote 91 lines to plan.md
    1 # Plan de mejora - `main.py` (alcance exacto del README)
    2
    3 ## Objetivo
    4 Reescribir `main.py` para pasar del modelo limitado (4 numéricas, solo accuracy, sin
    5 balanceo) al pipeline completo con 10 predictores, preprocesamiento por tipo de
    6 variable, `LogisticRegression` balanceada y métricas de la clase `1`, mostrando
    7 comparación ANTES vs DESPUÉS.
    8
    9 ## Evidencia del ANTES (ya ejecutada)
    10 ``
    ... +83 lines

● Guardado en plan.md (raiz del proyecto). Incluye los 8 pasos, la evidencia del ANTES ya capturada, los entregables afectados
y lo que queda fuera de alcance. No he tocado ningún otro archivo.

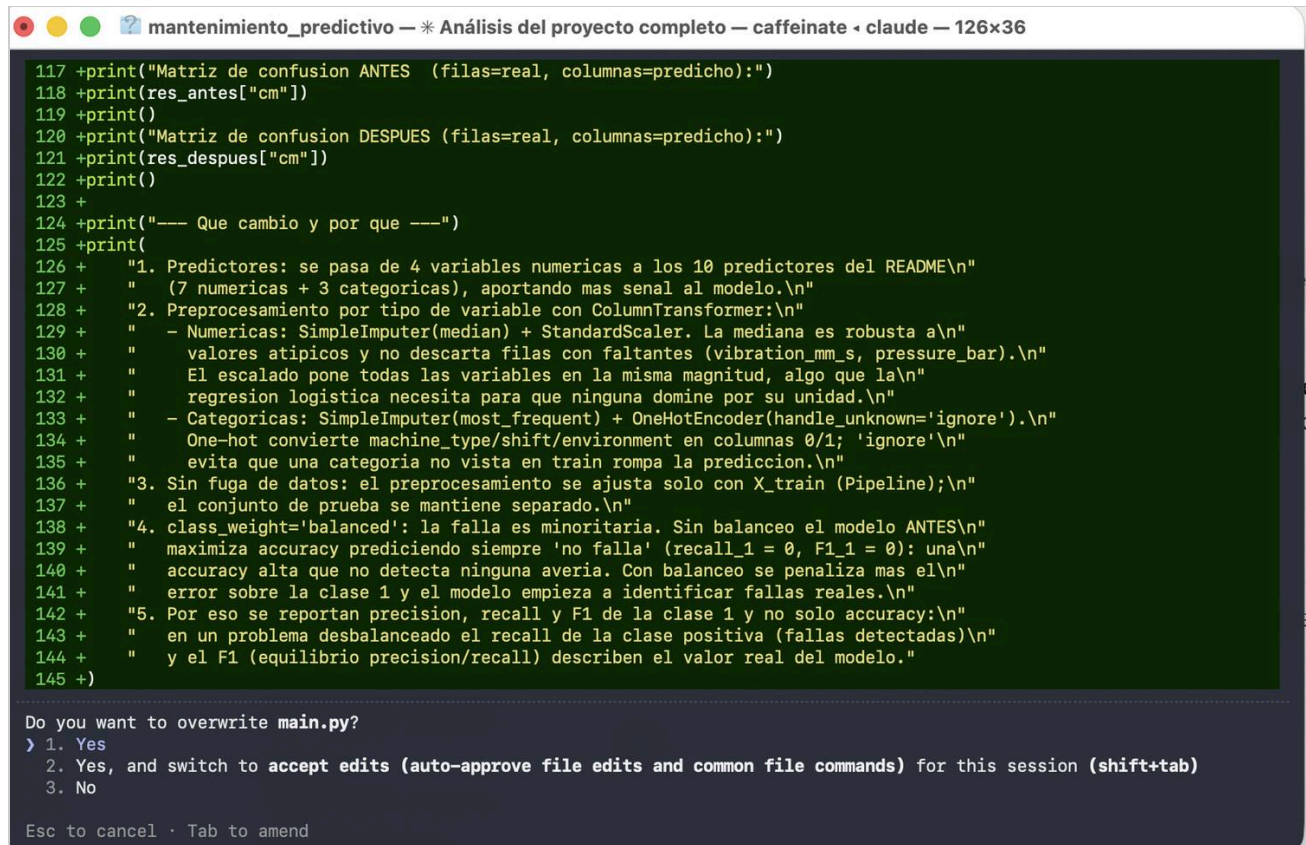
* Worked for 21s · done 2:19 PM
  Approve tool calls from your phone · https://claude.ai/code/session\_01QYUa4njw7EVKcxPbdE7S3f

> █

" manual mode on · ? for shortcuts · ← for agents /rc active
```

Figura 5

Paso 4: cambios propuestos sobre main.py, mostrados antes de autorizar la escritura.



```
117 +print("Matriz de confusion ANTES (filas=real, columnas=predicho):")
118 +print(res_antes["cm"])
119 +print()
120 +print("Matriz de confusion DESPUES (filas=real, columnas=predicho):")
121 +print(res_despues["cm"])
122 +print()
123 +
124 +print("—— Que cambio y por que ——")
125 +print(
126 +     "1. Predictores: se pasa de 4 variables numericas a los 10 predictores del README\n"
127 +     "   (7 numericas + 3 categoricas), aportando mas senal al modelo.\n"
128 +     "2. Preprocesamiento por tipo de variable con ColumnTransformer:\n"
129 +     "   - Numericas: SimpleImputer(median) + StandardScaler. La mediana es robusta a\n"
130 +     "     valores atipicos y no descarta filas con faltantes (vibration_mm_s, pressure_bar).\n"
131 +     "     El escalado pone todas las variables en la misma magnitud, algo que la\n"
132 +     "     regresion logistica necesita para que ninguna domine por su unidad.\n"
133 +     "   - Categoricas: SimpleImputer(most_frequent) + OneHotEncoder(handle_unknown='ignore').\n"
134 +     "     One-hot convierte machine_type/shift/environment en columnas 0/1; 'ignore'\n"
135 +     "     evita que una categoría no vista en train rompa la prediccion.\n"
136 +     "3. Sin fuga de datos: el preprocesamiento se ajusta solo con X_train (Pipeline);\n"
137 +     "     el conjunto de prueba se mantiene separado.\n"
138 +     "4. class_weight='balanced': la falla es minoritaria. Sin balanceo el modelo ANTES\n"
139 +     "     maximiza accuracy prediciendo siempre 'no falla' (recall_1 = 0, F1_1 = 0): una\n"
140 +     "     accuracy alta que no detecta ninguna averia. Con balanceo se penaliza mas el\n"
141 +     "     error sobre la clase 1 y el modelo empieza a identificar fallas reales.\n"
142 +     "5. Por eso se reportan precision, recall y F1 de la clase 1 y no solo accuracy:\n"
143 +     "     en un problema desbalanceado el recall de la clase positiva (fallas detectadas)\n"
144 +     "     y el F1 (equilibrio precision/recall) describen el valor real del modelo."
145 +)
```

Do you want to overwrite main.py?

- > 1. Yes
- 2. Yes, and switch to **accept edits** (auto-approve file edits and common file commands) for this session (shift+tab)
- 3. No

Esc to cancel · Tab to amend

Figura 6

Paso 5: ejecución posterior a la mejora. Evidencia del DESPUÉS.

```
mantenimiento_predictivo — * Análisis del proyecto completo — caffeinate · claude — 126x36

ANTES          DESPUÉS
[[1139         0]      [[795 344]
 [ 111         0]]      [ 49  62]]

Lectura

- El ANTES tenía 91 % de accuracy pero era inútil: predecía siempre "no falla", 0 fallas detectadas (recall 0, F1 0).
- El DESPUÉS baja la accuracy a 0.69 pero ahora detecta 62 de 111 fallas reales (recall 0.56). El coste son 344 falsos positivos (precision 0.15): revisa máquinas que no iban a fallar. En mantenimiento predictivo ese intercambio suele valer la pena —una inspección de más es más barata que una avería no anticipada—, y se puede ajustar moviendo el umbral de decisión si se quisiera (fuera de alcance).
- La caída de accuracy es esperada y correcta: class_weight="balanced" deja de premiar el "todo 0". Por eso el README pide mirar recall/F1 de la clase 1 y no la accuracy sola.

Qué contiene ahora main.py

1. 10 predictores (7 numéricas + 3 categóricas).
2. ColumnTransformer: rama numérica SimpleImputer(median) + StandardScaler; rama categórica SimpleImputer(most_frequent) + OneHotEncoder(handle_unknown="ignore").
3. Split test_size=0.25, random_state=42, stratify=y — el mismo para ambos modelos, así la comparación es justa.
4. LogisticRegression(max_iter=1000, random_state=42, class_weight="balanced").
5. Preprocesamiento ajustado solo con train (Pipeline).
6. Reporta accuracy + precision/recall/F1 de la clase 1 + ambas matrices de confusión + explicación en la salida.

Pendiente de tu parte (flujo del README)

- Guardar esta salida como evidencia del DESPUÉS.
- Completar BITACORA.md con tus palabras (no lo he tocado; el README pide que sea propio).

* Cogitated for 45s · done 2:21 PM

✓ Update installed · Restart to apply

> Completa BITACORA.md con la explicación del pipeline

" manual mode on · ? for shortcuts · ← for agents /rc active
```

Figura 7
Tabla comparativa de métricas ANTES vs DESPUÉS generada por el proyecto.

mantenimiento_predictivo

— * Análisis del proyecto completo — caffeinate

claude — 126x36

```
4. LogisticRegression(max_iter=1000, random_state=42, class_weight="balanced").
5. Preprocesamiento ajustado solo con train (Pipeline).
6. Reporta accuracy + precision/recall/F1 de la clase 1 + ambas matrices de confusión + explicación en la salida.

Pendiente de tu parte (flujo del README)

- Guardar esta salida como evidencia del DESPUÉS.
- Completar BITACORA.md con tus palabras (no lo he tocado; el README pide que sea propio).

* Cogitated for 45s · done 2:21 PM

> ejecuta python3 main.py y muestrame la salida

Ran 1 shell command

• Esa es la salida completa del DESPUÉS. Idéntica a la ejecución anterior (el resultado es determinista: random_state=42 fijo).

Resumen de métricas:
```

modelo	accuracy	precision (1)	recall (1)	F1 (1)
ANTES	0.9112	0.0000	0.0000	0.0000
DESPUÉS	0.6856	0.1527	0.5586	0.2398

```
Ya puedes guardar esta salida como evidencia del DESPUÉS. Queda pendiente completar BITACORA.md con tus palabras.

* Baked for 17s · done 2:22 PM

✓ Update installed · Restart to apply

> completa BITACORA.md con la explicación del pipeline

" manual mode on · ? for shortcuts · ← for agents

/rc active
```